

Documentation

Date:1/12/25

Objective: Setting up the project and create simple get and post api.

Steps Taken:

- 1.Download the intellij IDE along with java 17.
- 2.Download the spring initializr zip file for the project-with spring-web, thymeleaf dependencies.
- 3.Extract the file and put on intellij ide.
- 4.Create package for Controller ,Service ,Entity and Repository.
- 5.Create simple get and post api for tasks on controller and functions on service class.
- 6.Create simple thymeleaf html file on template folder for viewing and adding task.

Challenges :Errors when connecting with thymeleaf.

Verification: Program runs well after i corrected the error and run. All apis work well.

Date:3/12/25

Objective: Create delete and update .

Steps Taken:

- 1.Creating delete and update api endpoint in the controller class.
- 2.Creating delete and update function in service layer.

Challenges : N/A

Verification: Program runs well when i run the program.

Date:4/12/25

Objective: Add validating and start implementing css.

Steps Taken:

1. Adding validating dependency on pom file and add not null annotation on some attributes like title etc.
2. Start implementing css by creating css file on static css folder.

Challenges : N/A

Verification: Program runs well when i run the program.

Date:5/12/25

Objective: Mysql Db connection.

Steps Taken:

1. Downloading Mysql workbench for windows.
2. Adding jpa ,mysql driver on pom.xml file.
3. Adding mysql configuration on application.properties e.g password for my mysql.

Challenges : N/A

Verification: Program runs well when i run the program and db works well in fetching and putting data on db.

Date:10/12/25

Objective: Adding pagination,filter and sorting logic .

Steps Taken:

1. Adding pagination,filter and sorting logic on Controller view tasks followed by adding main logic on service class.
2. Adding customize jpa functions in repo class to enable pagination ,filter ,sorting logic.

Challenges : Errors getting in implementing these logics and coordination fails on excuting these logics at initial phase.

Verification: Program runs well when i run the program and getting desire pagination ,filter and sorting tasks logics.

Date:11/12/25

Objective: Adding Junit test cases with mockito.

Steps Taken:

- 1.Creating test function for each function from service class.
- 2.Mocking wherever necessary.
- 3.Creating +ve and -ve test cases.

Challenges : N/A

Verification: Test functions runs well and tested.

Date:15/12/25

Objective:1. Creating enum classes for status and priority tasks
2. Add completedAt attribute and mark as done button in frontend.

Steps Taken:

- 1.Creating enum package and create enum classes for status and priority.
- 2.Replacing String datatypes from previous state of my project with enums.
- 3.Adding completedAt attribute in entity class and implement logic when user click mark as done to show completed time.

Challenges : Error comes in initial phase.

Verification: Program runs well when i run the program after correcting error and getting desire output.

Date:16/12/25

Objective: Creating view task to view individual task.

Steps Taken:

- 1.Create view button on ui .
- 2.Implement logic for when user click view button on ui it has to view the task through id of task pass through pathvariable.

Challenges : N/A.

Verification: Program runs well when i run the program.

Date:17/12/25

Objective: Modifying search logic to handle ignore case search.

Steps Taken:

1. Adding Ignore case in findByTitle, findBydescription etc in repository class.

Challenges : N/A.

Verification: Program runs well when i run the program and getting desire output.

Date:18/12/25

Objective: Pagination, filter and sorting with ui.

Steps Taken:

1. Integrate backend pagination, filter and sorting logic with ui.

Challenges : Error comes in initial phase and does not get desire output at initial phase.

Verification: Program runs well when i run the program after correcting error and getting desire output.

Date:23/12/25

Objective: Creating new apis for 1.pending tasks for today.

2.tasks to do in 2 days or 5 days.

3.Overdue tasks.

Steps Taken:

1.Adding required api endpoints in Controller class and implement logic in service class.

2.Implement logic for these in repo class by implementing DueDateBetween etc.

Challenges : N/A

Verification: Program runs well when i run the program ,api works well even when testing on postman.

Date:24/12/25

Objective: Adding swagger ui for documenting api.

Steps Taken:

1.Adding swagger dependency with `springdoc-openapi` library.

Challenges : N/A

Verification: Api document can see well in `localhost:8080/swagger-ui`

.

Date:29/12/25

Objective: Creating Authentication and Authorization for my project.

Steps Taken:

1. Creating User Controller ,User Entity and UserService class for login, registration for user.
2. Creating otp verification frontend ui page after registration.

Challenges : N/A

Verification: It works well on running.

Date:30/12/25

Objective: Email api integration.

Steps Taken:

1. Adding email api dependency in pom.xml.
2. Creating dto for email.
3. Adding logic for simplemail sender to send otp for verification after registration with simple secure random function to generate otp and send through email.

Challenges : N/A

Verification: Getting otp through email and works well.

Date:5/1/26

Objective: Adding Session Management.

Steps Taken:

- 1.Add HttpSession in login function in controller class and service.
2. Add logic to delete when user click logout.
- 3.With these logic to display user name on ui.

Challenges : N/A

Verification:It works well when running .

Date:7/1/26

Objective: Adding fragment for header and footer.

Steps Taken:

- 1.Creating fragment folder under template and create header and footer html file.
- 2.Using this in other page with th:replace of thymeleaf.

Challenges : Ui breaks at first ,not getting desire fix header and footer.

Verification: It works after correction.

Date:8/1/26

Objective: Making features for creating tasks for each user and display tasks for each user.

Steps Taken:

- 1.Add ManytoOne foreign key for user in task entity.
- 2.Modify some logic in service class for these feature

Challenges : N/A

Verification: It works well on running.

Date:9/1/26

Objective: Adding notification as reminder for pending tasks using email.

Steps Taken:

- 1.Add scheduling logic by creating scheduler folder ,thereby creating scheduler class which implements scheduler using cron expression which sets to 9 am of every to send Tomorrow pending tasks.

Challenges : N/A

Verification: It works well on running.

Date:13/1/26

Objective: Adding password encryption and dashboard.

Steps Taken:

- 1.Create bcrypt password encoder bean and implement encrypt password and verifying password in service layer.
- 2.Create dashboard using js charts ,add more styling using css and interact with backend to display real data of tasks.

Challenges : Got some error at initial stage while implementing dashboard.

Verification: It works well on running after correcting the error.

Date:13/1/26

Objective: Implement user profile,profile page that can upload profile,remove profile, change profile.

Steps Taken:

- 1.Add profile image attribute with byte[] datatype.
- 2.Implement controller for profile endpoints like uploading profile,changing profile,etc and implement profile logic in service layer.
- 3.Making profile pic display on headers of every pages of my appl and when click goes to profile page in frontend.

Challenges : Got some error at initial stage.

Verification: It works well on running after thorough correction.

Date:19/1/26

Objective: Implement csv file downloadable or send via email for all tasks.

Steps Taken:

1. Implemented CSV generation logic using Apache Commons CSV to export all user tasks in memory.
2. Added a controller endpoint to download the generated CSV file directly via HTTP response.
3. Integrated email functionality to send the CSV file as an attachment using `JavaMailSender`.
4. Add export button in which when user click it have choice for download or send via email in frontend.

Challenges : N/A

Verification: It works well on running.

Date:20/1/26

Objective: Integration testing and add mit license.

Steps Taken:

1. Check overall working of my appl by checking every functionality on running.
2. Copy mit license from mit license website and paste on mit license txt file.

Challenges : N/A

Verification: It works well on running.

Date:22/1/26

Objective: Deploying project on aws.

Steps Taken:

- 1.**Prepare the application** – Configure production properties, build the project (`. jar`), and ensure the database and email configs are ready for deployment.
- 2.**Launch AWS infrastructure** – Create an EC2 instance, configure security groups (open required ports), and install Java, Git, and necessary dependencies.
- 3.**Deploy the application** – Upload the JAR file to EC2, set environment variables, and run the application using `java -jar`.
- 4.**Access & verify** – Assign a public IP or domain, test application endpoints, and verify logs to ensure the app runs successfully.

Challenges : N/A

Verification: It works well on running.